ResRec — คู่มือเข้าใจโปรเจกต์ทั้งระบบ สำหรับเตรียมตอบคำถามอาจารย์

Restaurant Recommendation From Reviews — NLP Group Project

ทีม	Deesampan Vongsurit (6510766), Wattanan Jiratriluk (6520214)
โค้ดที่อ้างอิง	commit `cb6945f` (main) — โค้ด โมเดล และ `data.js` เหมือน `cfa9c5f` ทุกบรรทัด ต่างแค่เอกสาร — เลขบรรทัด (ไฟล์:บรรทัด) ทุกจุดตรวจกับเวอร์ชันนี้แล้วด้วยสคริปต์
วันที่	16 ตุลาคม 2026 (ฉบับที่ 3 — repo บน GitHub เป็นสาธารณะแล้ว และ Vercel อัปเดตเป็นตัวเลขของโมเดลที่เทรนเองแล้ว)

> **⚠ เอกสารนี้ AI เป็นคนร่าง — ทีมต้องตรวจและเล่าด้วยคำพูดของตัวเอง**
> เอกสารนี้ Claude (AI) เขียนขึ้นจากการอ่านโค้ดจริงและรันทดสอบจริงในเครื่องนี้ ตัวเลขทุกตัวมาจากไฟล์ผลลัพธ์ใน repo หรือจากการรันจริง แต่ **นี่เป็นงานในรายวิชา** ทีมต้องอ่านทวน ตรวจว่าถูก และเรียบเรียงคำตอบเป็นภาษาของตัวเอง อย่านำข้อความนี้ไปส่งเป็นงานเขียนของตัวเองโดยไม่บอกอาจารย์

สารบัญ

0. สรุปทั้งโปรเจกต์ใน 1 หน้า (อ่านอันนี้ก่อน)
1. โปรเจกต์นี้ AI ทำ หรือเอามาจาก open source?
2. NLP ถูกใช้ในเว็บอย่างไร — ไล่จากข้อความรีวิวไปจนถึงหน้าเว็บ
3. การกำจัด "noise" — เกณฑ์ที่รีวิวจะถูกตัดทิ้ง
4. ข้อมูลไฟล์ไหนสร้างตัวเลขไหนบนเว็บ
5. วิธีพิสูจน์ว่าเว็บใช้ NLP จริง (ทดสอบแล้ว พร้อมผลก่อน/หลัง)
6. คำถามที่อาจารย์น่าจะถาม พร้อมแนวคำตอบ
7. เรื่องที่ยังค้างอยู่ (ควรรู้ก่อนพรีเซนต์)

0. สรุปทั้งโปรเจกต์ใน 1 หน้า

โปรเจกต์ทำอะไร: อ่านรีวิวลูกค้าจาก Zomato ของร้านอาหาร 3 ร้านในเมืองบังกาลอร์ (อินเดีย) รวม 1,400 รีวิว แล้วบอกว่าลูกค้า **ชอบหรือไม่ชอบเมนูไหน** ไม่ใช่แค่ชอบร้านหรือไม่

ไอเดียหลักที่ต้องพูดได้: รีวิวเดียวชมเมนูหนึ่งแต่ติอีกเมนูได้ เช่น *"Kebab was yummy but biryani was bland"* เราจึง **ตัดรีวิวเป็นอนุประโยค** (ตัดตามจุดและคำอย่าง but) หาชื่อเมนูในแต่ละอนุประโยค ให้**โมเดลที่เราเทรนเอง**อ่านว่าอนุประโยคนั้นชม ติ หรือไม่ได้บอก แล้วให้เมนูได้ความเห็นของอนุประโยคที่มันอยู่ จากนั้นนับว่าแต่ละเมนูมีคนชอบกี่คนจากทั้งหมดกี่คน (ทดสอบแล้ว: Kebab → ชอบ, Biryani → ไม่ชอบ)

> **ระบบมีอย่างละหนึ่ง**
>
> | เว็บ | ข้อมูลที่เว็บอ่าน | คำสั่งสร้างข้อมูล | ตัวอ่านความรู้สึก |
> |---|---|---|---|
> | `index.html` → `restaurant.html` | `output_project/web/data.js` | `python run_pipeline.py` | **TF-IDF + Logistic Regression** ที่เราเทรนเอง จาก 133,389 รีวิว (ไม่รวมรีวิวของ 3 ร้านบนเว็บ) |
>
> VADER (ตัววัดความรู้สึกสำเร็จรูป) เหลือไว้แค่เป็นตัวเปรียบเทียบตอนเทรน ไม่มี spaCy ไม่มี HuggingFace / BERT

- **ข้อมูล:** ชุดข้อมูลสาธารณะ "Zomato Bangalore Restaurants" บน Kaggle (ผู้เผยแพร่ Himanshu Poddar) — **ไม่ได้ scrape เอง**
- **โมเดล:** ทดสอบกับ 26,678 รีวิวที่โมเดลไม่เคยเห็น ถูก **82.2%** (VADER สำเร็จรูป 72.7%, เดาคำตอบที่พบบ่อยที่สุดอย่างเดียว 60.5%) ถ้าแยกแค่ชอบ/ไม่ชอบ ถูก 96.5%
- **ผลบนเว็บ:** รีวิวเชิงบวก Biriyani Nights 36%, Night Food Joint 26%, BOX8- Desi Meals 57% — มีเมนูที่ข้อมูลพอจัดอันดับ 8 / 9 / 17 เมนู
- **Noise:** ข้อมูลดิบ 1,317,782 รีวิว ซ้ำกันถึง 89% เหลือ 134,612 รีวิวที่สะอาด ส่วน 3 ร้านบนเว็บ 1,400 → 1,321 รีวิว (ข้อ 3)
- **เปิดดูเว็บ:** ในเครื่องด้วย `start.bat` (http://localhost:8000) หรือสำเนาออนไลน์ https://resrec.vercel.app — ตัวเลขชุดเดียวกัน (ตรวจแล้ว 16 ต.ค. 2026)
- **พิสูจน์ว่าใช้ NLP จริง (รันแล้ว):** แก้ข้อความรีวิวเดียว "gulab jamun is **mouth watering and tasty af!**" → "**stale and tasteless!**" โดยไม่แตะดาว Gulab Jamun ของ Biriyani Nights ตกจาก 8 ใน 11 (73%) เป็น 7 ใน 11 (64%) และตกลงไป 1 อันดับ (ข้อ 5)

1. โปรเจกต์นี้ AI ทำ หรือเอามาจาก open source?

คำตอบสั้น (ตามหลักฐาน)

> **โค้ดเขียนโดยใช้ AI ช่วย (Claude Code) ภายใต้การกำหนดของทีม ไม่ได้คัดลอกโปรเจกต์ open source มาทั้งก้อน** แต่ใช้ **ไลบรารีสำเร็จรูป** (scikit-learn, NLTK, RapidFuzz ฯลฯ) และ **ชุดข้อมูลสาธารณะ** จาก Kaggle ซึ่งอ้างอิงไว้ใน `output_project/reports/sources.md` ตัวโมเดลวิเคราะห์ความรู้สึก**เราเทรนเอง**
>
> ประโยคนี้ AI สรุปจากหลักฐานด้านล่าง — ทีมควรปรับให้ตรงกับความจริงว่าใครออกแบบ/ตัดสินใจ/ตรวจส่วนไหน

หลักฐานที่ตรวจแล้ว

ตรวจอะไร	พบอะไร
ประวัติ git (`git log`)	มี 7 commit จากบัญชีของทีม commit แรก `97d6c94` (7 ต.ค. 2026) ใส่ไฟล์ทั้งโปรเจกต์เข้ามาในครั้งเดียว ประวัติก่อนหน้านั้นไม่อยู่ใน repo จึง**ไล่ไม่ได้ว่าโค้ดเก่าแต่ละบรรทัดใครเขียน**
บรรทัด `Co-Authored-By: Claude`	**มีในทั้ง 4 commit ของวันที่ 16 ต.ค. 2026**: `3dd93b0` (ซ่อมไฟล์ raw และทำให้ run_pipeline รันได้), `cfa9c5f` (เปลี่ยนเว็บมาใช้โมเดลที่เทรนเอง ลบ classic.html), `f0fecd2` (เพิ่มเอกสารนี้) และ `cb6945f` (เอกสารเรื่อง repo สาธารณะ) — เป็นหลักฐานใน git ตรง ๆ ว่า AI ร่วมเขียน
ไฟล์ `CLAUDE.md` ใน repo	บรรทัดแรกบอกว่าเป็นไฟล์ตั้งค่า "AI agent" ที่ทำงานใน repo นี้ `CLAUDE.md:3` และให้ AI เป็น "CTO" ที่ทั้งวางแผน เขียนโค้ด และออกแบบเว็บ `CLAUDE.md:9`
ลักษณะโค้ด	เฉพาะเจาะจงกับข้อมูลชุดนี้มาก (ชื่อคอลัมน์ Zomato, ชื่อร้าน "Biriyani Nights" ในคอมเมนต์ `src/dish_extract.py:60`) ไม่มีหัวไฟล์ license ของโปรเจกต์อื่น → **ไม่ใช่โปรเจกต์ open source ที่นำเข้ามา**

แยกให้ชัด: อะไรของเรา อะไรของคนอื่น

ส่วน	ที่มา	ใช้ทำอะไร
ทีม + AI เขียนเอง		
โค้ดทั้งหมดใน `src/` และ `run_pipeline.py`	ทีมกำหนด / AI ช่วยเขียน	กฎทำความสะอาด, การหาเมนู, การตัดอนุประโยค, การผูกความเห็นกับเมนู, การนับคะแนน
หน้าเว็บ `index.html`, `restaurant.html`, `app.js`, `styles.css`	ทีมกำหนด / AI ช่วยเขียน	แสดงผล
ไฟล์โมเดล `output_project/models/sentiment_tfidf_lr.pkl`	**เราเทรนเอง** ด้วย scikit-learn จากรีวิว Zomato	บอกว่าข้อความชม / ติ / ไม่ได้บอก
ไลบรารีของคนอื่น (เวอร์ชันจาก `requirements.txt`)		
scikit-learn 1.5.2	อัลกอริทึม TF-IDF + Logistic Regression (อัลกอริทึมเป็นของห้องสมุด แต่ค่าน้ำหนักเราเทรนเอง)	โมเดลของเรา
NLTK 3.10.3	รายการ stopwords + ตัวตัดคำ `word_tokenize`	ขั้นตัดคำ
RapidFuzz 3.14.5	วัดความเหมือนของคำ	จับชื่อเมนูที่สะกดผิดเล็กน้อย
VADER (`vaderSentiment` 3.3.2) `requirements.txt:4`	Hutto & Gilbert 2014 — พจนานุกรมความรู้สึกสำเร็จรูป	**แค่ตัวเปรียบเทียบตอนเทรน** `src/train_sentiment.py:150-156` ไม่ได้ใช้บนเว็บแล้ว
ftfy, langdetect, pandas, numpy, Flask, Pillow, matplotlib, ฟอนต์ Poppins	ห้องสมุดทั่วไป	ซ่อมตัวอักษร, ตรวจภาษา, จัดการตาราง, เซิร์ฟเวอร์, รูป
ข้อมูลของคนอื่น		

Zomato Bangalore Restaurants	Kaggle, Himanshu Poddar `references/references.txt:1` — สิทธิ์ "Data files © Original Authors" `output_project/reports/sources.md:37` (ไม่ใช่ open licence)	รีวิวทั้งหมด
ที่ไม่ได้ใช้: spaCy และ HuggingFace `transformers` / BERT — CLAUDE.md เขียนไว้ชัด `CLAUDE.md:132`		

2. NLP ถูกใช้ในเว็บอย่างไร

คำสั่งเดียว `python run_pipeline.py` ทำครบ 6 ขั้นตามที่เสนอไว้ (เก็บรีวิว → ทำความสะอาด → หาเมนู → อ่านความรู้สึก → รวมคะแนน → ส่งให้เว็บ) ฟังก์ชันหลักอยู่ใน `run_pipeline.py:10-49`

ขั้น	ทำอะไร (ภาษาง่าย)	โค้ด
1. โหลด	อ่าน 1,400 รีวิวของ 3 ร้าน (แก้ตัวขึ้นบรรทัดที่ git บน Windows เพิ่มให้ เพื่อให้ผลเหมือนกันทุกเครื่อง) และอ่านรายการเมนูของแต่ละร้าน	`run_pipeline.py:12-13` `src/load_data.py:8-14` `src/common.py:12-13` `src/load_data.py:17-43`
2. ทำความสะอาด	ลบคำนำหน้า "RATED", ซ่อมตัวอักษรเพี้ยน, ตัด emoji, ตัด URL, `!!!` → `!`, `soooo` → `soo`, รวมบรรทัด	`src/preprocess.py:32-46`
	คัดรีวิวขยะทิ้ง 4 ข้อ (N1–N4) เก็บรายการที่ถูกทิ้งไว้แสดงบนเว็บ	`src/preprocess.py:84-87`
	ตัดคำด้วย NLTK เอา stopwords ออกแต่ **เก็บคำปฏิเสธ** (not, no, never) — ถ้าเครื่องยังไม่มีข้อมูล NLTK จะโหลดให้เองครั้งแรก	`src/preprocess.py:21-23` `src/preprocess.py:49-52` `src/preprocess.py:15-19`
3. หาเมนู	สร้าง "พจนานุกรมเมนู" ของแต่ละร้าน จากเมนูจริง + แท็ก dish_liked + เมนูทั่วไป 20 รายการ ไม่นับเครื่องดื่มอย่าง coke/pepsi และคำว่า combo	`src/dish_extract.py:27-43` `src/dish_extract.py:13-15` `src/dish_extract.py:32`
	ทำคำที่สะกดต่างกันให้เป็นคำเดียว (biriyani → biryani, kabab → kebab)	`src/common.py:17-31`
	ไล่หาชื่อเมนูในข้อความ: ชื่อยาวก่อน ("chicken biryani" ชนะ "biryani"), คำยาว ≥ 6 ตัวอักษรยอมให้สะกดผิดเล็กน้อย (ความเหมือน ≥ 88 ด้วย RapidFuzz), ไม่นับชื่อร้านเป็นเมนู	`src/dish_extract.py:54-78` `src/dish_extract.py:46-51` `src/dish_extract.py:16` `src/dish_extract.py:60-64`
4. ตัดอนุประโยค	ตัดหลัง `. ! ? ;` และก่อนคำ but / however / although / though / whereas / except / while	`src/sentiment.py:27` `src/sentiment.py:65-72`
5. ซ่อนคำก่อนให้โมเดลอ่าน	ซ่อน 3 อย่าง: **ชื่อเมนู** (ไม่ให้เมนูถูกตัดสินจากชื่อเสียงทั่วบังกาลอร์), **คำเรื่องการสั่ง** (ordered / order / tried / got — โมเดลเรียนมาว่า "ordered" เป็นคำลบ เพราะคนบ่นมักเริ่มด้วย "I ordered ...") และ **คำว่า but ที่ขึ้นต้น** (โมเดลเรียนมาว่าเป็นสัญญาณรีวิว 3 ดาว)	`src/sentiment.py:28-34` `src/sentiment.py:73-77`
6. อ่านความรู้สึก	โมเดลของเราให้คะแนน = P(บวก) − P(ลบ) ตั้งแต่ −1 ถึง +1 ข้อความที่ไม่มีคำที่โมเดลรู้จักเลย = "ไม่ได้บอกอะไร"	`src/sentiment.py:18-22` `src/sentiment.py:42-62` `src/common.py:61-64`
	อนุประโยคจะ "เลือกข้าง" ก็ต่อเมื่อโมเดลมั่นใจ: คะแนน ≥ +0.4 = ชอบ, ≤ −0.4 = ไม่ชอบ, ระหว่างนั้น = ไม่เลือกข้าง	`src/sentiment.py:35-39` `src/sentiment.py:78-80`
7. ผูกกับเมนู	เมนูได้ความเห็นของอนุประโยคที่มันอยู่ ถ้าอนุประโยคนั้นไม่ได้บอกอะไรเลย (มีแต่ชื่อเมนู / "I ordered ...") ให้ยืม **อนุประโยคถัดไป** เฉพาะเมื่ออนุประโยคถัดไปไม่มีเมนูอื่นและพูดถึงอาหาร (ไม่ใช่เรื่องส่งของ) — เกิดขึ้น 21 จาก 915 ครั้งที่พบชื่อเมนู	`src/sentiment.py:93-111` `src/sentiment.py:106-108` `src/sentiment.py:24-26`
8. รวมคะแนน	1 รีวิว = 1 เสียงต่อ 1 เมนู (ถ้าพูดถึงหลายครั้ง ใช้ครั้งสุดท้ายที่เลือกข้าง) เปอร์เซ็นต์คิดจากคนที่ **เลือกข้าง** เท่านั้น	`src/sentiment.py:114-117` `src/aggregate.py:35-61` `src/aggregate.py:53-56`
	ติดป้าย (ต้องมี ≥ 4 ความเห็น): ชอบ ≥ 80% = "Customers love it", ≤ 40% = "Needs work", 40–60% = "Mixed opinions"	`src/aggregate.py:7-10` `src/aggregate.py:23-32`
	"รีวิวเชิงบวก" ของร้าน = โมเดลอ่านทั้งรีวิว (ใช้คำตอบของโมเดลตรง ๆ ไม่ใช้กฎ 0.4) — **ไม่ได้มาจากดาว**	`src/sentiment.py:84-86` `run_pipeline.py:24` `src/aggregate.py:64-88`

9. ส่งออก	เขียน `web/data.js` (+ สำเนา `results/resrec_data.json`) และรีวิวที่ถูกทิ้งลง `web/removed.js`	`run_pipeline.py:43-49`
		`src/export_web.py:42-47`
		`src/export_removed.py:15-22`

10. หน้าเว็บ	หน้าโหลด `data.js` แล้ว `app.js` วาดการ์ดร้าน (% รีวิวบวก), ช่องสรุป (เมนูที่ชอบที่สุด / ถูกติที่สุด) และรายการเมนูเรียงจากชอบมากไปน้อย	`output_project/web/index.html:33-34`
		`output_project/web/restaurant.html:63-65`
		`output_project/web/app.js:70`
		`output_project/web/app.js:103-108`
		`output_project/web/app.js:113-123`

โมเดลมาจากไหน (`src/train_sentiment.py`)

- เทรนกับรีวิวที่ทำความสะอาดแล้ว (`reviews_clean.csv`) **ตัดรีวิวของ 3 ร้านบนเว็บออกก่อน** 1,223 รีวิว เหลือ 133,389 รีวิว `src/train_sentiment.py:54-68`
- แบ่ง เทรน 90,704 / ปรับค่า 16,007 / ทดสอบ 26,678 แบบสุ่มด้วย seed 42 `src/train_sentiment.py:165-170`
- TF-IDF คำเดี่ยวและคู่คำ + Logistic Regression `src/train_sentiment.py:71-84` โดยเอาคำปฏิเสธออกจาก stopwords `src/train_sentiment.py:45-49`
- บันทึกเป็น `output_project/models/sentiment_tfidf_lr.pkl` `src/train_sentiment.py:300-307`

`app.py` ทำอะไรบ้าง

- เสิร์ฟไฟล์ในโฟลเดอร์ `output_project/web/` `src/app.py:26-33` — ไม่มีการคำนวณบนเซิร์ฟเวอร์แล้ว
- ปิด cache ทุกคำตอบ เพื่อให้รีเฟรชแล้วเห็นผลที่สร้างใหม่ทันที `src/app.py:19-23`
- `start.bat` เรียกไฟล์นี้ `start.bat:15`
- สำเนาแบบ static บน Vercel: https://resrec.vercel.app — อัปโหลดโฟลเดอร์ `output_project/web` ด้วย `npx vercel deploy --prod` `CLAUDE.md:150-151` ตรวจแล้ว (16 ต.ค. 2026) ว่าเสิร์ฟ `data.js` เนื้อหาเดียวกับใน repo และ `/classic.html` ไม่มีแล้ว (404)

3. การกำจัด "noise"

> **ประโยคที่ควรพูดได้**
> "Noise ขึ้นกับงาน รีวิวที่บ่นเรื่องคนส่งของเป็นรีวิวที่ดี แต่ไม่มีประโยชน์กับการจัดอันดับ**เมนู** เราจึงไม่ลบรีวิวพวกนี้ทิ้ง แต่ไม่เอาไปนับคะแนนเมนู ส่วนของที่เป็นขยะจริง ๆ — รีวิวซ้ำ สแปม ข้อความว่าง ภาษาอื่น — เราลบทิ้งเป็นชั้น ๆ และนับไว้ทุกชั้นว่าทิ้งไปกี่แถว"

ชนิดของ noise ที่เจอในข้อมูลนี้ และวิธีจัดการ

ชนิด	ตัวอย่าง	จัดการโดย
รีวิวซ้ำ	Zomato ลงร้านเดียวกันหลายหมวด (Delivery, Dine-out ...) รีวิวชุดเดียวกันจึงซ้ำหลายรอบ	**ลบ**
สแปม / ข้อความสำเร็จรูป	ข้อความเดียวกันโพสต์ในหลายร้าน	**ลบ** (ถ้าซ้ำเกิน 5 ร้าน)
ว่าง / สั้นเกิน / ไม่มีตัวอักษร	มีแต่ "RATED" หรือ emoji	**ลบ**
ยาวผิดปกติ	รีวิวยาวหลายพันตัวอักษร	**ลบ** (เกิน 5,000 ตัวอักษร)
ไม่ใช่ภาษาอังกฤษ	อักษรฮินดี / กันนาดา	**ลบ**
เครื่องหมายของแพลตฟอร์ม	`RATED\n` นำหน้าเกือบทุกรีวิว	แก้ข้อความ (ไม่ลบแถว)
ตัวอักษรเพี้ยน (mojibake)	`â€™` แทน `'`	แก้ข้อความ
URL / อีเมล / เบอร์โทร	—	แก้ข้อความ
ตัวอักษรยืด, คำย่อ	`gooood`, `didn't`	แก้ข้อความ (`good`, `did not`)
สะกดหลายแบบ	biriyani / briyani / biryani, panner / paneer	รวมเป็นคำเดียว
คำที่ทำให้โมเดลเข้าใจผิดในอนุประโยคสั้น	ชื่อเมนู, "I ordered ...", but ที่ขึ้นต้น	ซ่อนก่อนให้โมเดลอ่าน (ข้อ 2 ชั้น 5)
ไม่พูดถึงเมนูเลย	"delivery was late"	**ไม่ลบ** — ยังนับใน % รีวิวบวกของร้าน แต่ไม่นับคะแนนเมนู

ระดับ 1 — ทำความสะอาดข้อมูลทั้งชุด (ใช้เทรนโมเดล) `src/build_corpus.py`

ลำดับมีความหมาย: ตัดซ้ำก่อน (จะได้ไม่เสียเวลาทำความสะอาดแถวที่จะทิ้ง) และกรองภาษาก่อนให้โมเดลภาษาอังกฤษอ่าน ตัวเลขมาจาก `output_project/results/cleaning_funnel.csv`

ขั้น	เกณฑ์ที่ทำให้รีวิวถูกทิ้ง	โค้ด	เหลือ	ทิ้ง
แถวใน CSV	—		51,717	
แตกเป็นรีวิว	แถวที่อ่าน `reviews_list` ไม่ได้ หรือรีวิวที่ข้อความว่าง	`src/build_corpus.py:75-95`	1,317,782	
ตัดซ้ำ	ร้านเดียวกัน (ชื่อ + ที่อยู่) และข้อความเหมือนกันทุกตัวอักษร (เทียบด้วย MD5)	`src/build_corpus.py:97-108`	143,789	1,173,993
สแปม	ข้อความเดียวกันปรากฏใน **มากกว่า 5 ร้าน**	`src/build_corpus.py:30` `src/build_corpus.py:112-113`	140,055	3,734
ความยาว / คุณภาพ	หลังทำความสะอาด **สั้นกว่า 15** หรือ **ยาวกว่า 5,000** ตัวอักษร หรือไม่มีตัวอักษรเลย	`src/build_corpus.py:28-29` `src/build_corpus.py:119-125`	137,290	2,765
ภาษา	ตัวอักษรที่ไม่ใช่ ASCII **เกิน 25%** หรือ langdetect บอกว่าไม่ใช่ "en" หรือตรวจไม่ได้	`src/build_corpus.py:130-141`	134,619	2,671
ป้ายคำตอบ	ไม่มีดาว (ใช้เป็นคำตอบตอนเทรนไม่ได้)	`src/build_corpus.py:146`	134,612	7

การแก้ข้อความ (ไม่ทิ้งแถว): regex ลบ URL / อีเมล / เบอร์โทร `src/build_corpus.py:36-38` ฟังก์ชัน `structural_clean` `src/build_corpus.py:54-65` และ `normalise` (ย่อตัวอักษรยืด ขยายคำย่อ) ซึ่ง**เป็นฟังก์ชันเดียวกับที่ใช้ตอนเว็บอ่านรีวิว** โมเดลจึงเห็นข้อความแบบเดียวกันทั้งตอนเทรนและตอนใช้ `src/build_corpus.py:23` ตั้ง seed ให้ langdetect ได้ผลเดิมทุกครั้ง `src/build_corpus.py:26`

ระดับ 2 — เลือก 3 ร้าน `src/make_subset.py`

จัดอันดับร้านตามจำนวน **รีวิวที่ไม่ซ้ำ** แล้วเลือกอันดับ 1–3 `src/make_subset.py:26-48` ระหว่างส่งออกตัดรีวิวซ้ำออก 5,405 รายการ `src/make_subset.py:82-87` เหลือ 1,400 รีวิว: Biriyani Nights 524, Night Food Joint 446, BOX8- Desi Meals 430

ระดับ 3 — กรองรีวิวของเว็บ `src/preprocess.py` (จาก 1,400 รีวิว)

ชั้น	เกณฑ์	โค้ด	ทิ้ง	เหลือ
N1 ว่างหลังทำความสะอาด	ไม่เหลือข้อความเลย (เช่นมีแต่ emoji)	`src/preprocess.py:84-87`	6	1,394
N2 ซ้ำ	ร้านเดียวกัน ข้อความหลังทำความสะอาดเหมือนกัน		3	1,391
N3 สั้นเกิน	น้อยกว่า 2 คำ		70	1,321
N4 ไม่ใช่อักษรละติน	ตัวอักษร ASCII น้อยกว่า 70% `src/preprocess.py:64-66`		0	1,321

รีวิวที่ถูกทิ้งแสดงบนหน้าร้านหัวข้อ "Reviews left out" `output_project/web/app.js:130-150` จาก 1,321 รีวิวที่เหลือ มี **416 รีวิวที่พูดถึงเมนู** (รวม 747 ความเห็นต่อเมนู) อีก 905 รีวิวไม่พูดถึงเมนู — นับใน % รีวิวบวกของร้านอย่างเดียว

ข้อมูลเสียที่ซ่อมแล้ว

ไฟล์เมนู `top3_restaurants_raw.csv` เคยถูกบันทึกผ่าน Excel ซึ่งตัดช่องที่ยาวเกิน ~32,767 ตัวอักษร ทำให้แถวแตกเป็น 12 แถว × 3,664 คอลัมน์ commit `3dd93b0` เพิ่มสคริปต์ซ่อมที่ต่อแถวกลับเป็น 17 คอลัมน์ เมนูและข้อมูลร้านกลับมาครบ `src/repair_raw_csv.py:28-42`

4. ข้อมูลไฟล์ไหนสร้างตัวเลขไหนบนเว็บ

ข้อมูลดิบ

ไฟล์	รายละเอียด
`data_raw/zomato.csv` `src/config.py:6-7`	**Zomato Bangalore Restaurants** — Kaggle, ผู้เผยแพร่ Himanshu Poddar, https://www.kaggle.com/datasets/himanshupoddar/zomato-bangalore-restaurants — 51,717 แถว สิทธิ์ "Data files © Original Authors" **หมายเหตุ:** ไฟล์นี้ **ไม่อยู่ใน repo** (ตั้งให้ git ไม่เก็บ `.gitignore:1`) และไม่มีในเครื่องนี้ ต้องโหลดจาก Kaggle ถ้าจะสร้าง `reviews_clean.csv` หรือเทรนโมเดลใหม่ — **แต่ไม่จำเป็นสำหรับการสร้างเว็บ**

ข้อมูลที่ได้จากการประมวลผล (`output_project/data/`)

ไฟล์ในตารางนี้ **อยู่ใน repo สาธารณะ** บน GitHub (ต่างจาก `zomato.csv` ที่ไม่ถูกเก็บ) — ดูคำถามที่ 15

ไฟล์	สร้างโดย	ใช้ที่ไหน
`reviews_clean.csv` (134,612 รีวิว)	`src/build_corpus.py`	เทรนโมเดล → `models/sentiment_tfidf_lr.pkl`
`top3_restaurants_reviews.csv` (1,400 รีวิว)	รีวิวของ 3 ร้าน (จาก `make_subset.py`)	**เว็บ** และใช้ตัดร้านเหล่านี้ออกจากข้อมูลเทรน
`top3_restaurants_raw.csv` (3 ร้าน, 7 แถว)	แถวดิบ Zomato (ซ่อมแล้ว)	**เว็บ**: เมนูและข้อมูลร้าน
`selected_restaurants_reviews.csv`, `dropped_duplicates.csv`, `dish_gazetteer.csv`, `sample_reviews.csv`	`make_subset.py` / `build_gazetteer.py`	**เว็บไม่ได้ใช้แล้ว** (เหลือจากเวอร์ชันก่อน)

แต่ละส่วนบนเว็บ ← ไฟล์ที่ใช้

สิ่งที่เห็นบนเว็บ	ไฟล์ที่หน้าเว็บอ่าน	ไฟล์ต้นทาง
หน้าแรก: การ์ด 3 ร้าน (% รีวิวบวก, จำนวนรีวิว)	`output_project/web/data.js`	`top3_restaurants_reviews.csv` + `top3_restaurants_raw.csv` + `models/sentiment_tfidf_lr.pkl` ผ่าน `run_pipeline.py`
หน้าร้าน: ช่องสรุป, แถบชอบ/ไม่ชอบของแต่ละเมนู, หน้าต่างรีวิวของเมนู (ไฮไลต์เขียว/แดง)		
หน้าร้าน: "Reviews left out"	`output_project/web/removed.js`	`top3_restaurants_reviews.csv` ผ่าน `src/export_removed.py`
รูปเมนู	`output_project/web/img/`	`src/image/`
รายงาน: confusion matrix	`output_project/figures/confusion_matrix.png`	`reviews_clean.csv` ผ่าน `src/train_sentiment.py`

5. วิธีพิสูจน์ว่าเว็บใช้ NLP จริง

หลักการ: แก้ **ข้อความ** ของรีวิวหนึ่งรีวิว โดยไม่แตะจำนวนดาว แล้วสร้างข้อมูลใหม่ ถ้าตัวเลขของเมนูเปลี่ยน แปลว่าตัวเลขมาจากการอ่านข้อความ ไม่ได้มาจากดาวหรือพิมพ์ใส่ไว้ตายตัว

รีวิวที่ใช้ทดสอบ

ร้าน **Biriyani Nights** รีวิว 4.5 ดาว (แถวที่ 928 เมื่อนับจาก 0) ท่อนที่แก้: *"The dessert that is **gulab jamun is mouth watering and tasty af!**"*

เปลี่ยนเป็น: `gulab jamun is stale and tasteless!` (ส่วนอื่นของรีวิวที่ชมบริยานียังอยู่เหมือนเดิม)

ขั้นตอน

1. เช็กว่า repo สะอาด:
   ```
   git status
   ```
2. เปิด `output_project\data\top3_restaurants_reviews.csv` ด้วย **VS Code หรือ Notepad (ห้ามใช้ Excel** — Excel ตัดช่องที่ยาวเกิน 32,767 ตัวอักษร แบบที่เคยทำไฟล์ raw เสีย)
3. ค้นหา `gulab jamun is mouth watering and tasty af!` → **Replace All** ด้วย `gulab jamun is stale and tasteless!` (เจอ 2 จุด ในแถวเดียวกัน คือคอลัมน์ข้อความดิบและข้อความที่ล้างแล้ว) แล้วบันทึก **ไม่แตะคอลัมน์ดาว**
4. สร้างข้อมูลใหม่ (ประมาณ 25 วินาที):
   ```
   .venv\Scripts\python.exe run_pipeline.py
   ```
5. เปิดเว็บ: ดับเบิลคลิก `start.bat` (หรือ `.venv\Scripts\python.exe src\app.py 8000`) → `http://localhost:8000` → คลิก **Biriyani Nights** แล้วกด **Ctrl+F5**
6. ดูแถว **Gulab Jamun** ในรายการเมนู
7. คืนค่าเดิม แล้วเช็กว่าสะอาด:
   ```
   git checkout -- output_project
   git status
   ```

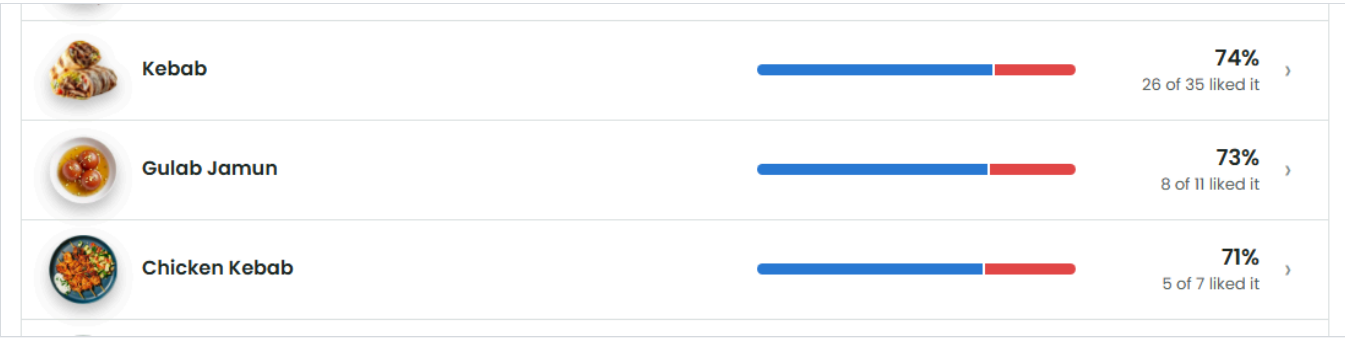
ผลที่ได้จริง (รันแล้ว 16 ต.ค. 2026)

สิ่งที่ดู	ก่อน	หลัง
Gulab Jamun (Biriyani Nights)	**73%** — 8 of 11 liked it	**64%** — 7 of 11 liked it
ลำดับในรายการเมนู	อยู่เหนือ Chicken Kebab	ตกลงไปอยู่ใต้ Chicken Kebab
% รีวิวบวกของร้าน	36%	36% (ไม่เปลี่ยน — ทั้งรีวิวยังชมบริยานีเป็นหลัก)
ดาวของรีวิวนี้	4.5	4.5 (ไม่ได้แตะ)

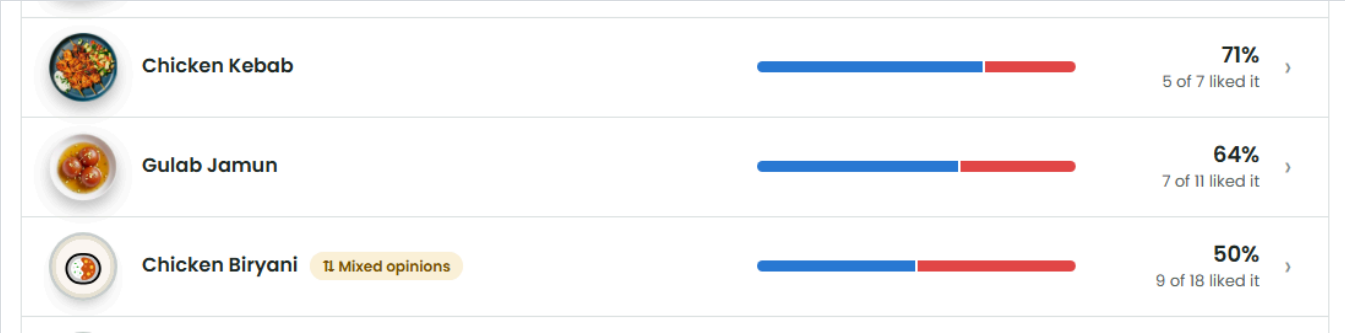
เปรียบเทียบผลทั้งหมด 127 รายการ (funnel, ค่าประเมิน, 3 ร้าน, 122 เมนู) **เปลี่ยนแค่รายการเดียว** คือ Gulab Jamun ของ Biriyani Nights — **ดาวไม่เปลี่ยน แปลว่าการเปลี่ยนมาจากข้อความล้วน ๆ**

ดูข้างในว่าโมเดลอ่านอนุประโยคนั้นอย่างไร (คะแนน = P(บวก) − P(ลบ)):

อนุประโยค	เมนูที่เจอ	คะแนน	ผล
The dessert that is gulab jamun is mouth watering and tasty af!	Gulab Jamun	+0.984	**ชอบ**
The dessert that is gulab jamun is stale and tasteless!	Gulab Jamun	−0.928	**ไม่ชอบ**



รูปที่ 1: restaurant.html?r=biriyani-nights **ก่อนแก้** — Gulab Jamun 73%, 8 of 11 liked it (อยู่ระหว่าง Kebab กับ Chicken Kebab)



รูปที่ 2: **หลังแก้** — Gulab Jamun 64%, 7 of 11 liked it แถบแดงยาวขึ้น และตกลงไปอยู่ใต้ Chicken Kebab

ทดสอบอย่างไร และคืนค่าเดิมแล้วหรือยัง

- ทำใน **สำเนาของ commit** `cfa9c5f` (โค้ดและข้อมูลเหมือน `cb6945f`) ในโฟลเดอร์ชั่วคราว ไม่ได้แก้ไฟล์ใน repo จริงเลย (`git status` สะอาด)
- ก่อนแก้ รัน `run_pipeline.py` กับข้อมูลเดิมแล้ว `data.js` ที่ได้ **ตรงกับที่ commit ไว้ทุกไบต์** (ต่างแค่ตัวขึ้นบรรทัดตัวสุดท้ายของไฟล์ ซึ่ง Windows เขียนเป็น CRLF)
- session หลักทดสอบรีวิวเดียวกันแยกต่างหากและได้ 8/11 → 7/11 เหมือนกัน
- รูปทั้ง 2 ถ่ายด้วย Chrome แบบ headless จากหน้าเว็บจริง แล้วครอปตรงแถวเมนู

6. คำถามที่อาจารย์น่าจะถาม พร้อมแนวคำตอบ

แนวคำตอบเหล่านี้ AI ร่าง — ทีมควรซ้อมพูดด้วยคำของตัวเอง ตัวเลขมาจาก `output_project/results/` , `report.md` หรือการรันจริง ตัวอย่างประโยคทุกตัวรันกับโมเดลปัจจุบันแล้ว

1. NLP อยู่ตรงไหนในโปรเจกต์นี้? ไม่ใช่แค่นับดาวเหรอ?

มี 4 งาน NLP: (1) ทำความสะอาดและตัดคำ (2) ตัดอนุประโยค (3) หาชื่อเมนูในข้อความ (4) จำแนกความรู้สึกด้วยโมเดลที่เทรนเอง ดาวใช้แค่เป็น "คำตอบ" ตอนเทรนโมเดล **ทุกตัวเลขบนเว็บมาจากโมเดลอ่านข้อความ** ทั้งคะแนนเมนูและ % รีวิวบวกของร้าน — พิสูจน์ได้ด้วยการทดสอบในข้อ 5 (เปลี่ยนข้อความ ดาวเท่าเดิม ตัวเลขเปลี่ยน)

2. Noise ในข้อมูลนี้คืออะไร เอาออกยังไง?

ใหญ่ที่สุดคือ **รีวิวซ้ำ 89%** (1,317,782 → 143,789) เพราะ Zomato ลงร้านเดียวกันหลายหมวด ตามด้วยสแปมที่ซ้ำเกิน 5 ร้าน (3,734), สั้น/ยาวผิดปกติ (2,765), ไม่ใช่ภาษาอังกฤษ (2,671) และไม่มีดาว (7) — เหลือ 134,612 นอกจากนี้แก้ข้อความ: ลบ "RATED", ซ่อมตัวอักษรเพี้ยน, ลบ URL/อีเมล/เบอร์, ย่อตัวอักษรยืด, ขยายคำย่อ รีวิวที่ไม่พูดถึงเมนูไม่ได้ลบ แต่ไม่นับคะแนนเมนู ทุกขั้นนับไว้ใน `cleaning_funnel.csv`

3. ทำไมไม่ใช้ ABSA (Aspect-Based Sentiment Analysis)?

ABSA ต้องมีข้อมูลที่คนติดป้ายระดับ "เมนู + ความรู้สึก" ซึ่งไม่มีในชุดนี้ (มีแต่ดาวระดับรีวิว) และซับซ้อนเกินกว่าจะอธิบายได้ในงานวิชานี้ เราใช้วิธีที่ง่ายกว่าแต่ได้ผลระดับเมนูเหมือนกัน: ตัดอนุประโยค → หาเมนู → ให้โมเดลอ่านอนุประโยค → ให้เมนูได้ผลนั้น ข้อแลกคือถ้าสองเมนูอยู่ในอนุประโยคเดียวกันจะได้ผลเดียวกัน

4. รีวิวที่ชมเมนูหนึ่งแต่ติอีกเมนู ระบบทำยังไง?

"Kebab was yummy but biryani was bland." ตัดก่อนคำว่า but ได้ 2 อนุประโยค: "Kebab was yummy" (+0.991 → Kebab ชอบ) และ "but biryani was bland." (−0.945 → Biryani ไม่ชอบ) ถูกทั้งคู่ `src/sentiment.py:27` ประโยค *"the salad was amazing, but the chicken was cold"* จะนับแค่ salad เพราะ "chicken" คำเดียวเป็นวัตถุดิบ ไม่ได้บอกว่าเมนูไหน — ตั้งใจออกแบบไว้

5. ถ้าอนุประโยคแค่บอกว่าสั่งอะไร เช่น "I ordered biryani. It was cold." ล่ะ?

"I ordered biryani." หลังซ่อนชื่อเมนูและคำว่า ordered แล้วไม่เหลือคำที่โมเดลรู้จัก จึง "ไม่ได้บอกอะไร" ระบบจึงยิมอนุประโยคถัดไป "It was cold." (−0.98) → Biryani ไม่ชอบ (ทดสอบแล้ว) `src/sentiment.py:93-111` เกิดแบบนี้ 21 จาก 915 ครั้งที่พบชื่อเมนู

6. ทำไมเลือก TF-IDF + Logistic Regression ไม่ใช้ BERT?

ง่าย อธิบายได้ เทรนบน CPU ได้ในไม่กี่นาที และ**น้ำหนักของคำคือคำอธิบาย**: คำที่ดันไปทางลบมากสุดคือ worst (+12.99), pathetic, not, bad; ทางบวกคือ awesome (+9.94), best, amazing, delicious ผลบน 26,678 รีวิวทดสอบ: **ถูก 82.2%, macro-F1 0.774** ดีกว่า VADER สำเร็จรูป (72.7%, 0.543) และดีกว่าเดาคำตอบที่พบบ่อยสุด (60.5%, 0.251) BERT น่าจะแม่นกว่า แต่หนักกว่า อธิบายยากกว่า และไม่จำเป็นต่อคำถาม "เมนูไหนดี"

7. คำตอบที่ใช้เทรน (label) มาจากไหน?

จากดาวของรีวิว: ≥ 4 = บวก, 3–3.5 = กลาง, < 3 = ลบ `src/build_corpus.py:44-51` เป็น "weak label" คือไม่ได้มีคนอ่านแล้วติดป้ายเอง แบ่งข้อมูลแบบสุ่มด้วย seed 42 ข้อจำกัด: เทรนกับรีวิวทั้งรีวิว แต่นำไปใช้กับอนุประโยคสั้น ๆ (จึงต้องมีกฎซ่อนคำและกฎ 0.4)

8. ทำไมต้องตัดรีวิวของ 3 ร้านบนเว็บออกจากข้อมูลเทรน?

ถ้าไม่ตัด โมเดลจะเคยเห็นรีวิวเหล่านี้พร้อมดาวมาแล้วตอนเทรน ผลบนเว็บก็จะดูดีเกินจริง และการเช็กว่า "โมเดลตรงกับดาวกี่ %" จะไม่ยุติธรรม เราจึงตัด 1,223 รีวิวของ 3 ร้านนี้ออกก่อนเทรน `src/train_sentiment.py:36-38` ผลบนรีวิวของเว็บจึงเป็นผลบนข้อมูลที่โมเดลไม่เคยเห็น

9. ระบบแม่นแค่ไหน? วัดความแม่นระดับเมนูหรือยัง?

ระดับรีวิว: ถูก 82.2% บนชุดทดสอบ; คลาส "กลาง" อ่อนสุด (F1 0.582) เพราะรีวิว 3 ดาวกำกวม ถ้าแยกแค่ชอบ/ไม่ชอบ ถูก 96.5% บนรีวิวของเว็บ (ที่โมเดลไม่เคยเห็น) ตรงกับดาว **90.1%** จาก 1,147 รีวิว (ไม่นับ 2.5–3.5 ดาว) เวลาบอกว่าบวกถูก 97.7% `src/aggregate.py:91-106` **ระดับเมนู/อนุประโยคยังไม่ได้วัดตรง ๆ** เพราะไม่มีป้ายระดับอนุประโยค — ใช้ดาวเช็กคร่าว ๆ: เมนูที่ถูกบอกว่า "ชอบ" ในรีวิว 1 ดาว หรือ "ไม่ชอบ" ในรีวิว 5 ดาว มี 21 จาก 747 ความเห็น ต้องยอมรับตรง ๆ และเสนอว่าถ้าทำต่อ จะติดป้ายอนุประโยคด้วยมือสัก 200 อัน

10. ทำไมต้องซ่อนชื่อเมนูและคำว่า "ordered"?

โมเดลเรียนจากรีวิวทั้งรีวิว คำบางคำจึงมีความหมายแฝง: "ordered" มีน้ำหนักไปทางลบ (+4.1) เพราะคนบ่นมักเล่าว่า "I ordered ..." ถ้าไม่ซ่อน "I ordered veg biryani." จะกลายเป็นคำบ่น ชื่อเมนูเองก็มี "ชื่อเสียง" จากทั้งบังกาลอร์ติดมา และ "but" ที่ขึ้นต้นเป็นสัญญาณรีวิว 3 ดาว (+3.78 ไปทาง neutral) `src/sentiment.py:28-34`

11. ทำไมใช้เกณฑ์ 0.4?

อนุประโยคสั้นกว่ารีวิวที่โมเดลเรียนมา จึงให้เลือกข้างเฉพาะตอนที่มั่นใจ ใช้ดาวเซ็กคร่าว ๆ: ความผิดชัด ๆ (ชอบในรีวิว 1 ดาว / ไม่ชอบในรีวิว 5 ดาว) ลดจาก 52 เหลือ 21 แลกกับความเห็นที่เลือกข้างลดลง (554 จาก 747) และเมนูที่จัดอันดับได้ลดจาก 40 เหลือ 34 **ข้อควรพูดตรง ๆ:** ค่า 0.4 เลือกด้วยการเซ็กเดียวกันนี้ ตัวเลขจึงเข้าข้างมัน — มันแสดงว่ากฎตัดความผิดที่ชัดเจนได้ ไม่ได้พิสูจน์ความแม่นระดับอนุประโยค

12. ทำไมไม่เอาคำว่า not / no ออกเหมือน stopword ทั่วไป?

เพราะ "not good" จะกลายเป็น "good" ทดลองแล้ว: เก็บคำปฏิเสธไว้ได้ผลดีกว่าใช้ stopword ปกติของ scikit-learn **+2.1 จุด accuracy** (80.1% → 82.2%) และ +2.5 จุด macro-F1 `src/train_sentiment.py:45-49`

13. หาเมนูด้วย NER ไหม?

ไม่ โมเดล NER สำเร็จรูป (เช่นของ spaCy) ไม่มีประเภท "อาหาร" มีแค่คน องค์กร สถานที่ วันที่ และการเทรน NER เองต้องมีข้อมูลที่ติดป้ายตำแหน่งชื่อเมนู ซึ่งเราไม่มี จึงใช้การเทียบกับพจนานุกรมเมนูของแต่ละร้าน + จับคำสะกดผิดเล็กน้อย `src/dish_extract.py:54-78` (ข้อเสนอเดิมเขียนว่าใช้ NER — ตอนนี้เปลี่ยนแล้ว และเอา spaCy ออกจากโปรเจกต์)

14. มีกรณีที่ระบบผิดไหม? ยกตัวอย่าง

มี ตัวอย่างจริงจากข้อมูล (เจอโดยหาเมนูที่ "ชอบ" ในรีวิว 1 ดาว):

- **ชมของอย่างอื่นในอนุประโยคเดียวกัน:** "Loved the colorful box that the sandwich wrap was packed in." (รีวิว 1 ดาว) → Sandwich และ Wrap ถูกนับว่า **ชอบ** ทั้งที่ชมกล่อง
- **คำบรรยายของที่สั่งฟังดูดี:** "We ordered home made dum biriyani and butter chicken." (รีวิว 1 ดาว) → ทั้งสองเมนู **ชอบ** เพราะโมเดลเรียนมาว่าคำว่า "home" (+2.43) และ "dum" (+1.16) เป็นคำบวก
- **การเปรียบเทียบ:** "but the price is like Paradise biriyani." → Biryani **ชอบ** (Paradise เป็นร้านบริยานีชื่อดัง)
- **"not bad":** "The biryani was not bad." ได้ −0.076 → ไม่เลือกข้าง ทั้งที่คนพูดค่อนไปทางชม
- **ชื่อเมนูต้องตรงกับเมนูร้าน:** "lemonade was not upto mark" ที่ BOX8 ไม่ถูกนับ เพราะเมนูร้านเขียน "Masala Lemonade"
- สองเมนูในอนุประโยคเดียวได้ผลเดียวกัน; ภาษาผสม (Hinglish) และคำสะกดผิดมาก ๆ อาจหาเมนูไม่เจอ

15. ใช้ข้อมูลนี้ได้ถูกต้องตามสิทธิ์ไหม?

ต้องตอบตามจริง: ข้อมูลอยู่บน Kaggle สิทธิ์ "Data files © Original Authors" ซึ่ง**ไม่ใช่ open licence** repo บน GitHub (`deesampan/nlp_project`) **เป็นสาธารณะ** ตั้งแต่ 16 ต.ค. 2026 ตามการตัดสินใจของทีม ใน repo **ไม่มีไฟล์ดิบ** `zomato.csv` (`data_raw/` ไม่ถูกเก็บใน git) แต่**มีไฟล์ที่ได้จากการประมวลผลซึ่งมีข้อความรีวิวอยู่**: `reviews_clean.csv` (134,612 รีวิว), รีวิวของ 3 ร้าน และ `data.js` ของเว็บ (ซึ่ง Vercel ก็เสิร์ฟแบบสาธารณะ ~1,300 รีวิว) การเผยแพร่แบบนี้เกินกว่าที่สิทธิ์อนุญาตชัดเจน **ทีมรู้และยอมรับความเสี่ยงนี้เอง** และเขียนไว้ใน `sources.md` `output_project/reports/sources.md:52-56` เราอ้างอิงแหล่งที่มาครบและใช้เพื่อการศึกษา ถ้าอาจารย์เห็นว่าไม่ควร ทีมทำ repo กลับเป็น private ได้

16. ทำไมไม่ scrape รีวิวเอง?

เป็นการเปลี่ยนขอบเขตที่ตกลงไว้: scrape ผิดเงื่อนไขของแพลตฟอร์ม ทำซ้ำไม่ได้ (ข้อมูลเปลี่ยนตลอด) และเสียเวลาไปกับงานที่ไม่ใช่ NLP ชุดข้อมูลสาธารณะทำให้คนอื่นตรวจผลซ้ำได้

17. เลือก 3 ร้านนี้เพราะอะไร?

เป็น 3 ร้านที่มี **รีวิวไม่ซ้ำมากที่สุด** ในชุดข้อมูล `src/make_subset.py:26-48` ยิ่งรีวิวเยอะ คะแนนต่อเมนูยิ่งน่าเชื่อ ลำดับ % รีวิวบวก (57% > 36% > 26%) ตรงกับคะแนนบน Zomato เอง (4.1 > 3.2 > 2.9) ทั้งที่เราไม่ได้ใช้คะแนนนั้นเลย — เป็นการเซ็กแยกอีกทาง

18. ทำไมเมนูต้องมีความเห็นขั้นต่ำก่อนขึ้นอันดับ? ความเห็น "กลาง" ไปไหน?

กัน "1 จาก 1 = 100%" ขึ้นเป็นเมนูที่ดีที่สุด ต้องมี ≥ 4 ความเห็นที่เลือกข้าง `src/aggregate.py:7` เมนูที่น้อยกว่านี้แสดงแยกว่า "ข้อมูลยังน้อยเกินไป" ส่วนรีวิวที่แค่เอ่ยชื่อเมนูโดยไม่เลือกข้าง ไม่ถูกนับใน % (ตัวหารคือคนที่บอกชัดว่าชอบหรือไม่ชอบ) `src/aggregate.py:53-56`

19. AI เขียนโค้ดให้หรือเปล่า?

ตอบตามจริง (ดูข้อ 1): ใช้ AI coding assistant (Claude Code) ช่วยเขียนโค้ด มี `CLAUDE.md` และบรรทัด Co-Authored-By ใน commit เป็นหลักฐาน ทีมเป็นคนกำหนดโจทย์และขอบเขต (เช่น ตัด ABSA, ไม่ scrape, ใช้โมเดลเดียว) และต้องอธิบายได้ทุกขั้น — **เอกสารนี้มีไว้ให้ทีมเข้าใจโค้ดจนอธิบายได้เอง** (ทีมปรับประโยคนี้ให้ตรงกับความจริง)

20. รันซ้ำแล้วได้ผลเดิมไหม?

ได้ รัน `run_pipeline.py` จาก commit `cfa9c5f` ได้ `data.js` ตรงกับที่ commit ไว้ทุกไบต์ (ตรวจแล้ว; `cb6945f` ใช้โค้ดและ `data.js` ชุดเดียวกัน) ตั้ง seed ไว้ทั้งการแบ่งข้อมูล (42) และ langdetect (0) ครั้งแรกบนเครื่องใหม่ต้องต่ออินเทอร์เน็ตเพื่อโหลดข้อมูล NLTK ส่วนการเทรนโมเดลใหม่ต้องมี `zomato.csv` จาก Kaggle

7. เรื่องที่ยังค้างอยู่

ปัญหาที่ฉบับแรกเจอ **แก้แล้ว**ใน `3dd93b0` และ `cfa9c5f` : ไฟล์ raw เสีย, ข้อมูล NLTK, เครื่องวิเคราะห์ 2 ชุด, เอกสารไม่ตรงกับโค้ด ที่ยังเหลือ (ตรวจกับ `cb6945f` แล้ว):

#	เรื่อง	รายละเอียด / ผลกระทบ
1	`make_subset.py` ยังอ้างถึงไฟล์ที่ถูกลบ	บอกให้รัน `build_slice.py` ต่อ `src/make_subset.py:10` `src/make_subset.py:145` และเขียน `selected_restaurants_reviews.csv` แต่เว็บอ่าน `top3_restaurants_reviews.csv` → ถ้าจะเปลี่ยนร้าน ต้องคัดลอกไฟล์เอง
2	ไฟล์ที่เว็บไม่ได้ใช้แล้ว	`dish_gazetteer.csv` , `build_gazetteer.py` , `selected_restaurants_reviews.csv` , `dropped_duplicates.csv` , `sample_reviews.csv` — ถ้าอาจารย์เปิดดู ควรบอกได้ว่าเป็นของเวอร์ชันก่อน
3	ความแม่นระดับอนุประโยคยังไม่ได้วัด	มีแค่การเช็กคร่าวด้วยตาว (21 จาก 747) — ดูคำถามที่ 9, 11
4	Repo สาธารณะและเว็บ Vercel มีข้อความรีวิว	repo บน GitHub เปิดสาธารณะ มี `reviews_clean.csv` (134,612 รีวิว), รีวิวของ 3 ร้าน และ `data.js` ; เว็บ Vercel ก็เสิร์ฟ `data.js` (~1,300 รีวิว) ขณะที่สิทธิ์ข้อมูลไม่ใช่ open licence — **ทีมตัดสินใจยอมรับความเสี่ยงนี้เอง** `CLAUDE.md:152-155` ต้องตอบตรง ๆ ถ้าถูกถาม (คำถามที่ 15)
5	ไอคอนบางเมนูเป็นกล่องว่างบน Windows 10	emoji รูปแผ่นแป้ง (U+1FAD3) ของ Paratha, Naan, Roti ไม่มีในฟอนต์ของ Windows 10 จึงขึ้นเป็นกล่อง เห็นในภาพหน้าจอระหว่างทดสอบ
6	ต้องมีอินเทอร์เน็ตครั้งแรก	ข้อมูล NLTK โหลดอัตโนมัติตอนรันครั้งแรก `src/preprocess.py:14-19` ถ้าเครื่องกรรมการออฟไลน์ `run_pipeline.py` จะล้ม (แต่ `start.bat` เปิดเว็บได้ปกติ เพราะไม่ต้องสร้างข้อมูลใหม่)

จบเอกสาร — ร่างโดย AI (Claude) จาก commit cb6945f และการรันทดสอบจริงวันที่ 16 ต.ค. 2026 ทีมต้องตรวจทานก่อนใช้